EXPERIMENT 15: Write a Google App Engine Program to Display the Nth Largest Number from a List and Deploy It to Google Cloud

1. Aim

To develop a Python web application using Google App Engine to find and display the Nth largest number from a given list and deploy it to Google Cloud.

2. Procedure

1. Open Google Cloud Console and sign in.
2. Create a new Google Cloud project and select it.
3. Enable billing if required.
4. Install Python and Google Cloud CLI on the local computer.
5. Initialize the CLI using gcloud init and select the project.
6. Create a directory named NthLargest.
7. Inside the directory, create main.py.
8. Write a Flask application that accepts a comma-separated list and the value N.
9. Convert the entered values into integers and validate the value of N.
10. Sort the list in descending order.
11. Select the element at index N - 1 as the Nth largest element.
12. Create requirements.txt containing Flask.
13. Create app.yaml and specify the supported App Engine runtime.
14. Deploy the application using gcloud app deploy.
15. Open the application using gcloud app browse and test it with sample values.

3. Program

File – main.py:

```
from flask import Flask, request

app = Flask(__name__)

@app.route("/")
def largest():
    numbers = request.args.get("numbers")
    n = request.args.get("n")

    if numbers is None or n is None:
        return '''
        <h2>Nth Largest Number</h2>
        <form>
          Enter numbers (comma separated):
          <input name="numbers" required><br><br>
          Enter N:
          <input type="number" name="n" min="1" required><br><br>
          <input type="submit" value="Find">
        </form>
        '''

    try:
        arr = [int(x.strip()) for x in numbers.split(",")]
        n = int(n)

        if n < 1 or n > len(arr):
```

```
            return "Invalid N value"

        arr.sort(reverse=True)
        result = arr[n - 1]
        return f"<h2>{n}th Largest Number = {result}</h2>"

    except ValueError:
        return "Enter valid numbers"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=8080)
```

File – requirements.txt:

```
Flask
```

File – app.yaml:

```
runtime: python313
```

Deployment commands:

```
gcloud init
gcloud app deploy
gcloud app browse
```

4. Output of Program

```
Input numbers: 10, 50, 30, 20, 40
N = 2

Output:
2nd Largest Number = 40
```

Note: This program treats repeated values as separate list elements. For example, [50, 50, 40] has 50 as both the first and second largest elements under this convention.

5. Result

Thus, a Python web application to find the Nth largest number from a given list was developed and deployed using Google App Engine.

EXPERIMENT 15: Write a Google App Engine Program to Display the Nth Largest Number from a List and Deploy It to Google Cloud

1. Aim

To develop a Python-based web application using the Flask framework to identify and display the Nth largest number from a user-provided list and deploy the application using Google App Engine.

2. Procedure

1. Open a web browser and navigate to the Google Cloud Console. 2. Sign in using a valid Google account and create a new Google Cloud project. 3. Select the created project and enable the required Google Cloud services. 4. Enable billing if it is required for the selected deployment configuration. 5. Install Python and the Google Cloud CLI on the local computer. 6. Open a terminal or command prompt and execute gcloud init to initialize the Google Cloud CLI. 7. Authenticate using the Google account and select the appropriate Google Cloud project. 8. Create a new application directory named NthLargest. 9. Inside the directory, create a Python source file named main.py. 10. Import Flask and the request module to handle browser requests. 11. Create a Flask application instance and define a route for the home page. 12. Create an HTML form that accepts a comma-separated list of numbers and the value N. 13. Retrieve the submitted values using the Flask request object. 14. Split the comma-separated input into individual values and convert them into integers. 15. Validate the input to ensure that all entered values are valid integers. 16. Validate N to ensure that it is greater than or equal to 1 and does not exceed the number of elements in the list. 17. Sort the list in descending order using the sorting function. 18. Select the element at index N - 1 to identify the Nth largest element. 19. Display the calculated result on the browser using an appropriate HTML response. 20. Create a file named requirements.txt and include Flask as the application dependency. 21. Create an app.yaml file and specify the supported Python runtime. 22. Save all the application files in the same project directory. 23. Open the terminal inside the application directory and execute gcloud app deploy. 24. Wait until the deployment process completes successfully. 25. Execute gcloud app browse to open the deployed application. 26. Enter a sample list of numbers and a value for N. 27. Verify that the application displays the correct Nth largest number.

4. Output of Program

Sample Input: Input numbers: 10, 50, 30, 20, 40 N = 2 Expected Browser Output: 2nd Largest Number = 40 Explanation: The program sorts the entered numbers in descending order: 50, 40, 30, 20, 10 The second element in the sorted list is 40. Therefore, the second largest number is 40. The application validates the input and displays an error message if the entered numbers or N value are invalid. Note: The program treats repeated values as separate elements. For example, in the list [50, 50, 40], the second largest element is also 50 under this convention.

5. Result

Thus, a Python web application was successfully developed using Flask to identify the Nth largest number from a given list. The application was deployed on Google App Engine and tested through a web browser, producing the expected result. # Quick Revision – Important Commands Experiment Important commands GCC installation sudo apt update, sudo apt install gcc -y, gcc --version C compilation gcc file.c -o program, ./program Hadoop installation java -version, hadoop version Hadoop services hdfs namenode -format, start-dfs.sh, start-yarn.sh, jps Google App Engine gcloud init, gcloud app deploy, gcloud app browse Docker installation docker --version, docker info Docker image docker pull ubuntu, docker images Docker container docker run -it ubuntu bash, docker ps -a Nginx container docker run -d --name mynginx -p 8080:80 nginx VM file transfer scp sample.txt user@:/home/user/ File verification ls, cat sample.txt ## Important Points to Remember for the Practical Examination 1. Virtual Machines: A hypervisor creates and manages virtual machines by allocating physical hardware resources to each virtual environment. 2. GCC: GCC is a compiler that converts C source code into executable machine code. 3. Fibonacci Series: Each term is obtained by adding the previous two terms. 4. Google App Engine: It is a Platform as a Service (PaaS) that allows developers to deploy applications without directly managing the underlying server infrastructure. 5. Hadoop: HDFS provides distributed

storage, while YARN manages resources and job execution. 6. CloudSim: It is a simulation toolkit used to model cloud infrastructure, virtual machines, cloudlets and scheduling algorithms. 7. Docker Image: An image is a read-only template used to create containers. 8. Docker Container: A container is an isolated environment in which an application and its dependencies can execute. 9. Guest Additions: VirtualBox Guest Additions provides improved integration between host and guest operating systems. 10. SSH and SCP: SSH provides secure remote access, while SCP transfers files securely between networked systems. 11. Flask: Flask is a lightweight Python web framework used to develop web applications. 12. App Engine Deployment: The main.py, requirements.txt and app.yaml files should be maintained in the application directory. 13. Hadoop Configuration: Java and Hadoop environment variables must be configured correctly before starting Hadoop services. 14. Docker Port Mapping: Port mapping allows applications running inside containers to be accessed through ports on the host computer. 15. File Handling in C: fopen(), fprintf(), fgets() and fclose() are commonly used for file operations in C. Exam tip: For every experiment, clearly write the Aim, number the Procedure steps, include the complete program or commands, show a sample input and output, and finish with a Result beginning with “Thus…”. This keeps the answer consistent and easy to evaluate.